

Software Architecture

Basic Architecture Concepts

Bearbeitungshinweise

Ingo Arnold

2025

Inhaltsverzeichnis

1	Component.....	3
2	Interface.....	4
3	Composition	4

1 Component

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und konzentrieren Sie sich auf das Verständnis des Unterschieds zwischen Klassen, Objekten und Komponenten sowie deren Rolle in der Architektur. Sie sollten in der Lage sein, die Komponenten-Orientierung von der Objekt-Orientierung zu differenzieren sowie zu erklären, inwiefern die Komponenten-Orientierung komplementär zur Objekt-Orientierung ist.

Beantworten Sie darüber hinaus die folgenden Fragen:

- Welche Gemeinsamkeiten (und Unterschiede) weisen die in diesem Kapitel gegebenen Component-Definitionen auf?
- Warum reichen Klassen allein nicht aus, um komplexe Softwaresysteme strukturiert zu gestalten?
- Welche Vorteile bieten Komponenten gegenüber Klassen/Objekten im Hinblick auf Wartbarkeit und Wiederverwendbarkeit?

Was ist die zentrale Eigenschaft einer Software-Komponente?

- A. Sie enthält immer mindestens eine Datenbankverbindung.
- B. Sie ist ein isolierter Teil eines Systems mit klar definierten Schnittstellen.
- C. Sie kann nur innerhalb eines einzelnen Prozesses genutzt werden.
- D. Sie ersetzt automatisch jede andere Komponente im System.

Welche der folgenden Aussagen beschreibt am besten den Vorteil von Komponenten?

- A. Komponenten sind immer schneller als einzelne Klassen.
- B. Komponenten verhindern jede Form von Abhängigkeiten.
- C. Komponenten erhöhen die Wiederverwendbarkeit durch klare Abgrenzung.
- D. Komponenten ersetzen automatisch das gesamte Systemdesign.

Was ist KEIN typisches Merkmal einer Komponente?

- A. Kapselung von Funktionalität.
- B. Definierte Schnittstellen.
- C. Austauschbarkeit innerhalb eines Systems.
- D. Direkter Zugriff auf alle anderen Komponenten ohne Beschränkung.

Weitere nützliche Informationen und Erläuterungen finden Sie in:

- *Software Architecture: A Comprehensive Framework and Guide for Practitioners* [Vogel, Arnold et al 2011] (aka: SWA-Buch) – Kapitel:
 - 6.2.3 Component Orientation

- den unter [Bibliography](#) aufgeführten Literaturhinweisen im Foliensatz.

2 Interface

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und beantworten Sie die folgenden Fragen:

- Warum unterscheidet man zwischen Interfaces und dessen Implementierung im Zusammenhang mit Komponenten-Orientierung?
- Welche Aspekte und Arten der „Bindung“ existieren, wenn zwei Komponenten über ein entsprechendes Interface miteinander kooperieren, bzw. aneinander gebunden sind?

Wozu dient ein Interface in der Softwarearchitektur?

- Es implementiert die konkrete Logik einer Komponente.
- Es kapselt interne Datenbanktabellen vor den Entwicklern.
- Es definiert die erlaubten Zugriffsmöglichkeiten auf eine Komponente.
- Es sorgt dafür, dass eine Komponente nur einmal instanziiert werden kann.

Warum sind Interfaces wichtig für die Kopplung von Komponenten?

- Sie reduzieren die Abhängigkeit von konkreten Implementierungen.
- Sie beschleunigen den Compiler bei der Codegenerierung.
- Sie ermöglichen automatische Datenbankabfragen.
- Sie stellen sicher, dass jede Komponente eine grafische Oberfläche besitzt.

Was passiert, wenn eine Komponente ihr Interface verändert?

- Andere Komponenten, die das Interface nutzen, sind eventuell betroffen.
- Das gesamte System kompiliert automatisch neu ohne Probleme.
- Alle anderen Komponenten bleiben unabhängig und unverändert.
- Nur die interne Implementierung der Komponente wird angepasst.

Weitere nützliche Informationen und Erläuterungen finden Sie in:

- den unter [Bibliography](#) aufgeführten Literaturhinweisen im Foliensatz.

3 Composition

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und machen Sie sich das Schema des Aufbaus einer Komponente in Java (ohne, dass Component-Orientierung in Java mit eigenen Sprachmitteln unterstützt wird) so genau bewusst, dass

Sie dieses für die Konstruktion eigener Komponenten in Java jederzeit adaptieren könnten.

Seien Sie darüber hinaus in der Lage, das Façade Muster und dessen Einsatzbereich zu erklären und überlegen Sie sich – über die generische Anwendung des Musters in den Unterlagen hinaus – einen konkreten Fall, in dem Ihnen die Anwendung des Façade Musters gute Dienste leistet.

Nutzen Sie ggf. ChatGPT oder andere Quellen, um zentrale Konzepte weiter zu vertiefen.